

BORROWHUB

Build the product.
Understand the platform.

React Native + React | TypeScript BFF | Java Spring Boot | Azure + Bicep

Prepared for Levi Govender • 21 September 2026 • Version 1.0

Build a workplace equipment booking platform while learning how mobile clients, web clients, backend services and Azure infrastructure fit together. This is an implementation specification and learning guide, not an already-built application. All tasks begin as planned; no deployments, tests or application code have been completed by this document.

The default design uses separate React Native and React applications, one TypeScript BFF and one modular Spring Boot backend. A universal frontend remains an explicit decision gate. Architecture choices below are project recommendations; official documentation references appear at the end.

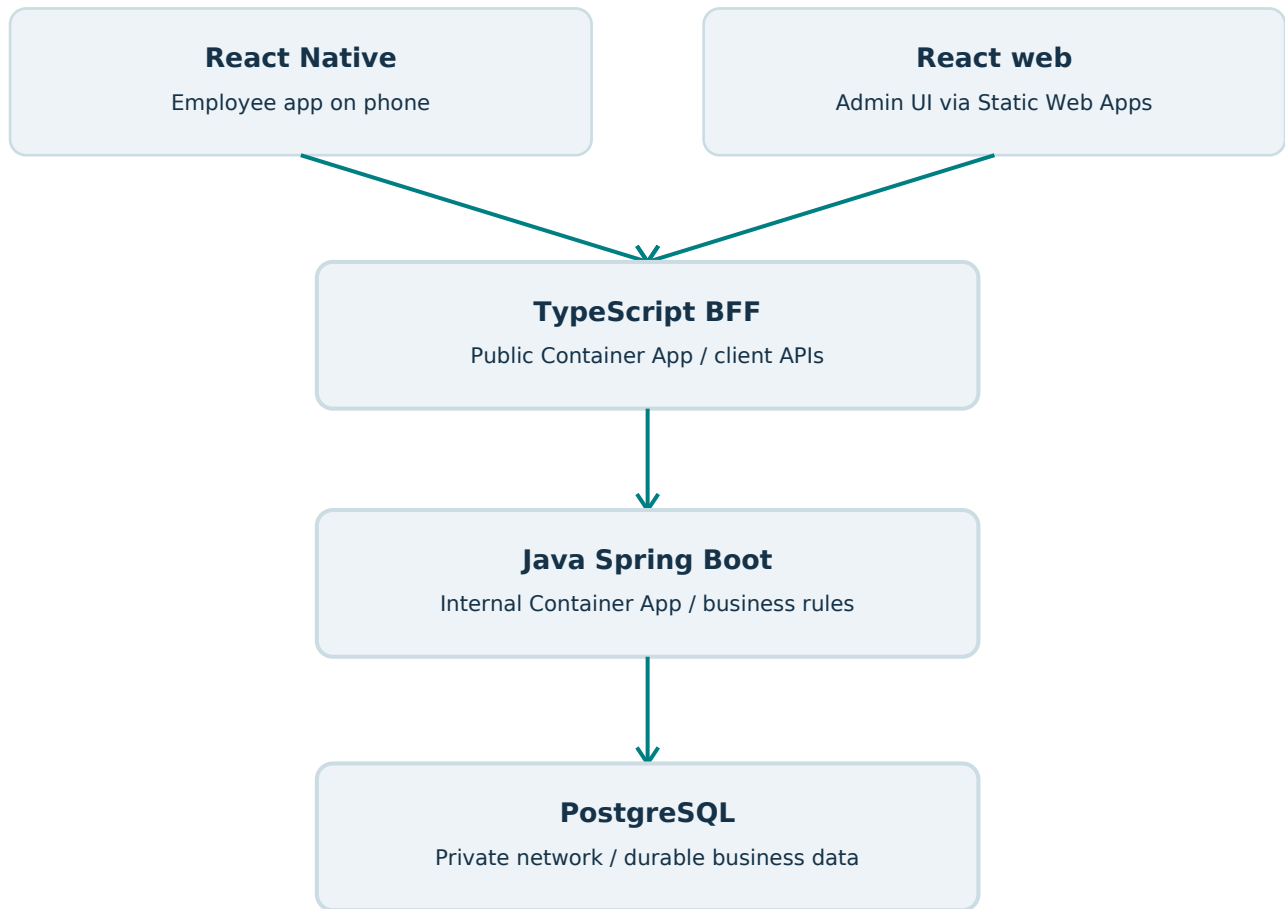
Use this PDF to understand the system. Put the companion Markdown files in the repository so Cursor can read the same specification, implement one task at a time and update progress with evidence.

MVP specification • Azure field guide • Cursor delivery workflow

Contents

01 / Product goals and boundaries	4
02 / Roles, journeys and product rules	5
03 / System architecture and boundaries	6
04 / MVP feature catalogue: employee experience	7
05 / MVP feature catalogue: operations	8
06 / Screens, navigation and universal decision	9
07 / TypeScript BFF implementation	10
08 / Spring Boot implementation	11
09 / Data model and invariants	12
10 / Correctness: concurrency and retry handling	13
11 / API contracts and example payloads	14
12 / Authentication and authorization, step by step	15
13 / Azure fundamentals: what lives where	16
14 / Containers, registry and Container Apps	17
15 / PostgreSQL, networking and DNS	18
16 / Managed identity, Key Vault and configuration	19
17 / Web hosting, logs, traces and costs	20
18 / Bicep: infrastructure as code in practice	21
19 / Bootstrap and CI/CD deployment order	22
20 / Local development and repository layout	23
21 / Test strategy and MVP completion gates	24
22 / Delivery plan with exit criteria	25
23 / Enhancement roadmap: better day-to-day use	26
24 / Enhancement roadmap: reliability and richer policy	27
25 / Cursor operating contract and progress tracking	28
26 / Troubleshooting and final demo checklist	29
27 / Official references and reading order	30

Architecture at a glance



Entra ID authenticates users; BFF exchanges delegated tokens for Java.

Bicep provisions resources. CI builds images, migrates and deploys.

ACR stores images. Key Vault holds secrets. Azure Monitor collects telemetry.

The app-to-app path is the product request path. Infrastructure provisioning, identity and telemetry support it but do not own booking rules.

01 / Product goals and boundaries

What you are building

BorrowHub lets employees find and reserve individual workplace assets such as test phones, monitors, adapters and cameras. Employees use a mobile application. Office administrators use a browser dashboard to manage assets and oversee loans. Each reservation refers to one specific physical asset, not a pooled quantity of interchangeable items.

The central workflow is browse, select a time window, reserve, collect, return. The most valuable technical problem is keeping inventory and reservations correct when people act concurrently or a network request is retried.

Learning outcomes

- Explain why the client calls a BFF and why the BFF does not own booking rules.
- Build accessible React and React Native screens that handle real API loading, empty, error and retry states.
- Implement Java domain rules, transactions, authorization and reliable database migrations.
- Package the BFF and backend as separate container images and deploy them to Azure Container Apps.
- Describe and reproduce infrastructure through Bicep rather than undocumented portal clicks.
- Trace a failed user action through client, BFF, backend and database using one correlation identifier.
- Demonstrate the complete system, automated checks and a documented deployment/rollback procedure.

MVP boundaries

One organisation, one office, one timezone for office policy, two application roles, one asset per booking and no payments. A user can hold multiple bookings for different assets. No booking approvals, waitlists, recurring bookings, push notifications, image uploads or offline mutations are required for MVP. Asset images use a bundled placeholder initially.

The mobile app is required for employees. The React web app is required for admins. The MVP includes manual collection and return buttons; scanning QR codes is the first enhancement. Public app-store release is outside MVP: development builds or approved internal distribution are sufficient.

Success criteria

Two test employees cannot successfully reserve the same asset for an overlapping interval. A user cannot access or mutate another user's booking by changing an ID. An admin can identify overdue loans. A clean checkout can run locally from documented instructions, and the cloud environment can be provisioned from source-controlled Bicep plus documented identity/bootstrap prerequisites.

02 / Roles, journeys and product rules

Employee

Signs in, searches inventory, checks availability, creates a reservation, views their bookings, cancels a future reservation, collects an asset and records its return. Employees cannot change inventory or inspect other employees' booking details.

Administrator

Has the employee capabilities plus inventory editing, all-booking search and operational actions. An admin may cancel or complete another user's booking, but must supply a reason recorded in the audit log. The user interface exposes only permitted actions; the Java backend enforces permissions independently.

Demonstration journey

1. Admin creates asset PHONE-001, a test phone in the equipment cupboard.
2. Employee A reserves it tomorrow from 09:00 to 12:00 office time.
3. Employee B tries 10:00 to 11:00 and sees a conflict; 12:00 to 13:00 is allowed.
4. Employee A collects it during the permitted collection window and later returns it.
5. Admin sees the completed booking and its audit events.
6. Repeat a checkout request to demonstrate safe handling of retries.

Proposed policy defaults

Reservations last at least 15 minutes and at most 7 days. A new reservation must start in the future and within the next 30 days. Collection is allowed from 15 minutes before the start until the reserved end, provided the asset is physically available. Cancellation is allowed only while RESERVED and before its start; an admin can cancel an uncollected reservation later with a reason. Editing times is outside MVP: cancel and create another booking.

These are product defaults, not Azure constraints. Keep them in backend configuration and test them using an injected clock. Show the policy in both clients; backend validation is authoritative. Use Africa/Johannesburg as the initial office timezone, UTC instants in storage and ISO 8601 timestamps with offsets in API requests.

Uncollected and late equipment

An expired, uncollected reservation cannot be checked out. Display it as expired when state is RESERVED and end is in the past; explicit expiry processing can be added later. A checked-out booking past its end is overdue, derived from state and time. Do not store OVERDUE as a separate lifecycle state. A late return may prevent the next customer from collecting; show that conflict and offer admin assistance instead of pretending the asset is physically present.

03 / System architecture and boundaries

Runtime topology

The React web bundle is served by Azure Static Web Apps. React Native runs on the phone. Both clients make HTTPS requests to the public TypeScript BFF. The BFF calls the internal Spring Boot Container App. Spring Boot alone owns relational data in PostgreSQL. Microsoft Entra ID authenticates users. Later, Spring Boot mediates access to private Blob Storage.

The BFF and Java service are separate Azure Container Apps in the same Container Apps environment. Give the BFF external ingress and the Java service internal ingress. An external environment can host an app with internal-only ingress; do not confuse an app's ingress setting with making the whole environment private. A private database additionally needs VNet integration and DNS configuration. [R1, R2, R7]

Ownership contract

- Clients: presentation, navigation, client-side form assistance and temporary UI state.
- BFF: client-facing API contracts, request validation at the boundary, identity token exchange, aggregation, timeouts and error translation.
- Spring Boot: domain validation, authorization, transitions, transactions, persistence and audit events.
- PostgreSQL: durable rows, keys, constraints and transactional locking.
- Azure: managed hosting, networking, identities, storage and operational tooling.
- Bicep: declared resource configuration. It does not execute booking logic or replace CI/CD.

Why a BFF is useful here

A phone home screen needs current loans, upcoming reservations and a small equipment summary. The BFF can fetch those backend resources concurrently and return one compact response. The admin screen needs counts, filters and a paginated table. It receives a different response shaped for that screen. Use one BFF deployment with `/api/v1/mobile` and `/api/v1/admin` routes initially.

Straightforward booking commands may be nearly pass-through. Do not manufacture extra transformations to justify the layer. The BFF has no database for booking records and cannot decide whether a booking is valid.

Scope protection

Use a modular monolith in Java, with packages for identity, equipment, booking and audit. Do not introduce microservices, Kubernetes, Redis, API Management or an event bus in MVP. Add another component only when a feature or measured constraint requires it.

04 / MVP feature catalogue: employee experience

MVP-01 / Sign-in and profile

Goal: reliably identify the employee and their permissions. Use single-tenant Microsoft Entra sign-in. The mobile app opens the system browser; the web app uses its browser authentication library. After sign-in, call `/api/v1/me` to obtain the application profile and allowed capabilities. Store stable tenant/object identifiers, not email as the user key.

Acceptance: unauthenticated API calls return 401; authenticated users see their name; sign-out clears local tokens/cache and user-specific query state. An expired session prompts sign-in without losing non-sensitive form input. Tokens and passwords are never logged. Local demo identity is allowed only in a clearly isolated development profile and must fail closed in cloud builds.

MVP-02 / Catalogue and asset details

Show asset name, asset tag, category, location and operational status. Provide text search, category filter and paginated results. The detail screen includes a description, booking policy and requested time-window availability. Operational status and availability are distinct: an ACTIVE item may already be reserved.

Acceptance: users can search by name or tag, reset filters, see an empty state and retry a failed request. Archived assets do not appear in employee search. A stale availability result never guarantees a later booking: creation revalidates everything in a transaction.

MVP-03 / Create reservation

The employee selects start/end times, reviews the asset and submits. Disable repeat taps while the request is pending and send an Idempotency-Key. The backend derives the owner from authentication; it ignores or rejects any supplied owner ID. On success, show a confirmed booking reference. On conflict, retain the chosen times and invite the user to choose another interval.

Acceptance: invalid dates produce field errors; overlapping bookings return 409; adjacent bookings are allowed; network retry does not create a second booking. A changed payload with the same idempotency key is rejected.

MVP-04 / My bookings and cancellation

Provide Upcoming, Current and History views. Detail shows asset, interval, state and available actions. Employees can only see their own records. Cancellation requires confirmation and an optional reason. Keep the cancelled record for history.

Acceptance: cancellation removes the time-window conflict atomically, leaves an audit record and refreshes relevant cached availability. Another user's booking is inaccessible even when its UUID is known. Repeating a successful cancel returns the same result or a documented harmless response.

05 / MVP feature catalogue: operations

MVP-05 / Collection and return

Collection changes RESERVED to CHECKED_OUT and records collectedAt. Return changes CHECKED_OUT to RETURNED and records returnedAt. Employees can perform these actions only on their own bookings; an admin can act on their behalf with a reason. Manual asset-tag confirmation is sufficient for MVP. This is a learning project's self-service policy; a real office may require staff verification.

Acceptance: collection outside the allowed window fails with a useful error. A second physical loan for the same asset cannot be collected concurrently. A return cannot precede collection. Repeated successful action requests do not duplicate audit events. Return includes a basic condition note, with full damage handling deferred.

MVP-06 / Admin inventory

Create and edit name, unique asset tag, category, description and location. Operational states are ACTIVE, MAINTENANCE and ARCHIVED. Do not hard-delete assets referenced by bookings. Use optimistic concurrency with a version field on admin edits so stale screens cannot silently overwrite newer changes.

Acceptance: duplicate tags return 409; stale edits return 409; employees receive 403 for admin mutations. Moving an asset out of ACTIVE is blocked if it has a checked-out or future active reservation. MVP tells the admin which bookings need resolution first, avoiding silent cancellations. Archived assets remain visible in historical booking records.

MVP-07 / Admin bookings and overdue dashboard

Show a paginated table with asset, borrower, interval, lifecycle state and overdue indicator. Filter by state, date range, asset and overdue. Include simple counts for current loans, upcoming bookings and overdue loans. Defer a drag-and-drop calendar until after MVP.

Acceptance: counts use the same backend rules as lists. Employees cannot access the table. Admin actions require a reason when acting for someone else. Pagination is stable using an explicit secondary sort such as ID.

MVP-08 / Audit and usable failure handling

Write an audit event for inventory changes and booking transitions inside the same database transaction as the mutation. Record actor, action, entity ID, timestamp, correlation ID and a minimal change summary. Admins can inspect an entity's history. This application audit is not the same as Azure's infrastructure activity log.

Acceptance: a failed transaction creates no success audit event. All screens have loading, empty and error states. API errors carry a stable machine code and trace ID, but never SQL text, stack traces or secrets. Core mobile controls support screen readers; web forms have labels and keyboard focus.

06 / Screens, navigation and universal decision

Mobile screen inventory

Sign-in; Home; Catalogue; Asset detail; Booking form; Booking confirmation/detail; My bookings; Profile. Home shows current loans, next reservations and quick access to browsing. Booking detail exposes cancel, collect or return based on capabilities from the API. A forbidden action must still fail safely if the client becomes stale.

Use React Native with TypeScript and an Expo development-build workflow as the proposed starting point. Expo provides tooling around native builds and device APIs; React Native renders native platform UI. Confirm work constraints before adopting Expo. Authentication redirect handling and native integrations may require a development build rather than Expo Go. [R14]

Web screen inventory

Sign-in; Dashboard; Inventory table; Asset create/edit; Bookings table; Booking detail and audit. Use React with TypeScript and Vite as the proposed web setup. React components render in the browser; Vite handles development and bundles deployable static assets. Client route guards improve navigation but do not provide backend security.

Use a query cache for server data and a small form library plus shared validation schemas for user input. Keep business rules server-side. Invalidate booking and availability queries after mutations; do not show a booking as confirmed before the server accepts it. Use separate credentials/caches for signed-in users.

Shared code contract

Share API-generated TypeScript types, API clients, date formatting, validation helpers, design tokens and permission labels. Do not import Java classes into TypeScript. Avoid sharing platform-specific token storage, navigation, QR scanning or DOM/native UI components unless a clear adapter exists.

DEC-01 / Universal frontend gate

Default: separate React web and React Native applications in a monorepo. Before implementation is far advanced, prototype a catalogue/detail screen in both and a representative admin table. A universal approach means using React Native-compatible components on web, commonly through React Native Web; it does not automatically make all packages or layouts portable.

Choose universal only if required libraries, authentication redirects, accessibility, responsive layout and admin table behaviour work acceptably. Record the result in DECISIONS.md. If selected, replace the two frontend task tracks with shared screens plus web/native adapters; backend contracts, acceptance criteria and cloud service boundaries stay the same. Do not silently change the decision while implementing.

07 / TypeScript BFF implementation

Runtime and structure

Run Node.js with TypeScript compiled to JavaScript in its own Docker image. Proposed framework: Fastify for explicit route schemas and a small surface area; NestJS is a reasonable substitute if the team prefers its conventions. Pick one during setup, record it and pin compatible versions.

Organise code into routes, schemas, auth, backendClient, aggregators and error handling. Keep route handlers small. Parse and validate input, retrieve the caller's downstream token, call typed Java API clients, map the result and return. Set a request body size limit and validate path/query parameters.

Aggregation example

GET /api/v1/mobile/home calls backend /v1/me, /v1/bookings/mine?view=current and /v1/bookings/mine?view=upcoming. Return a compact object with profile, currentLoans and upcomingBookings. The required profile must succeed; optional summary sections may return an explicit unavailable marker if you choose partial responses. Never silently substitute zero for a failed count.

GET /api/v1/admin/overview aggregates backend booking counts and an overdue preview. Aggregation may run independent requests concurrently with a bounded total timeout. Avoid one downstream call per asset row; Java should expose bulk or joined reads when needed.

Failure and reliability policy

Start with a 10-second total BFF request budget and shorter backend timeouts; tune from measured development behaviour, including cold starts. Retry reads at most once for selected transient failures with jitter. Never automatically retry a mutation unless its idempotency semantics are implemented. Return 503 for dependency unavailability and 504 for a timeout; map backend business conflicts to 409.

Propagate trace context and a validated/generated correlation ID. Log route, duration, status and trace ID, but not access tokens or full request bodies. Use structured JSON logs. Expose a minimal liveness endpoint and a readiness endpoint; do not expose internal diagnostics publicly.

Deliberate limitations

The BFF does not access PostgreSQL, calculate overlap rules or persist booking state. For MVP the browser uses bearer tokens rather than a server-side session-cookie BFF. A cookie-based browser session is a later security architecture option requiring CSRF controls and a shared session/token cache when scaled. The name BFF alone does not imply tokens are hidden from JavaScript.

08 / Spring Boot implementation

Application layers

Controllers accept DTOs, validate request shape and delegate to services. Services apply business rules and transaction boundaries. Repositories read/write PostgreSQL using Spring Data JPA. Configure JWT validation explicitly in Spring Security. [R16] Entities model persistence and never become public API responses directly. Mapping DTOs prevents accidental exposure of internal fields and lazy-loaded relationships.

Suggested dependencies: Spring Web, Validation, Data JPA, Security OAuth2 Resource Server, Actuator, PostgreSQL driver and Flyway. Add test dependencies for JUnit, Spring integration tests and Testcontainers. Choose a supported Java LTS/Spring Boot pair at implementation time and pin versions in the build wrapper; this guide does not assume an unverified latest version.

Module layout

identity resolves the current principal and application role. equipment owns inventory and operational status. booking owns reservations, state transitions and locking. audit records immutable application events. Modules can call services through explicit interfaces; do not let unrelated controllers reach into each other's repositories.

Transactions and business errors

Place @Transactional around each mutation service. Lock the asset row first for reservation creation, collection, cancellation, return and status changes; then load/check affected bookings in a consistent order. Persist state and audit together. Convert domain exceptions to a documented JSON error response through a controller advice. Validation failures use 400; authentication 401; permission failures 403; missing or hidden resources 404; conflicts 409.

Use DTO constraints for syntax such as required fields and maximum text length. Enforce temporal rules, ownership and lifecycle transitions in the service. An app role displayed by the client is not authority. Avoid logging sensitive personal data through entity toString methods.

Schema migration and health

Flyway applies versioned SQL migrations, such as V001__initial_schema.sql. Never modify a migration already applied to a shared database; add a new one. Set Hibernate schema management to validate in cloud environments, not create/update. Seed demo assets only under a development seed command/profile.

Expose limited health checks through Actuator. Liveness means the process is functioning; readiness means it can serve traffic. A database outage should not provoke endless liveness restarts. Keep detailed actuator endpoints internal. Application graceful shutdown should allow in-flight requests to finish within the platform termination budget.

09 / Data model and invariants

Core tables

AppUser: id UUID; tenantId; objectId; displayName; email for display; createdAt. Unique (tenantId, objectId). The signed token establishes identity; user-supplied email does not.

Equipment: id UUID; assetTag unique; name; category; description; location; operationalStatus; version; createdAt; updatedAt. Each row is one physical item. A version column supports optimistic locking for admin edits.

Booking: id UUID; equipmentId FK; userId FK; startAt and endAt timestampz; status; collectedAt; returnedAt; cancelledAt; cancellationReason; version; createdAt; updatedAt. Add checks for endAt greater than startAt and valid states. Preserve foreign keys and historical records.

AuditEvent: id UUID; actorUserId; entityType; entityId; action; occurredAt; correlationId; changeSummary JSON. Restrict application endpoints to reading events; create them through mutation services only. Avoid duplicating unnecessary personal information in the payload.

IdempotencyRecord: id; userId; route; key; requestHash; responseStatus; responseBody; createdAt; expiresAt. Unique (userId, route, key). Store the mutation result in the same transaction; retain for a proposed 24 hours and clean up later. Document that replay protection expires after the retention window.

State machine

RESERVED can become CHECKED_OUT or CANCELLED. CHECKED_OUT can become RETURNED. RETURNED and CANCELLED are terminal. Expired and overdue are display classifications, not extra persisted states. An admin does not bypass state integrity; exceptional corrections need a future dedicated, audited workflow.

Time and availability

Use half-open intervals [start, end). Two intervals overlap when $\text{existing.start} < \text{requested.end}$ AND $\text{existing.end} > \text{requested.start}$. Thus a booking ending at 12:00 does not block one starting at 12:00. Booking writes consider RESERVED and CHECKED_OUT intervals; cancelled and returned bookings no longer reserve future time. A checked-out asset also blocks another collection regardless of its scheduled end.

Future availability is a scheduling promise, not certainty about physical return. Show currently-on-loan status separately. The backend uses server time for policy decisions and converts user-entered office times to instants.

Index starting points

Index bookings by (equipmentId, startAt), (userId, startAt), and (status, endAt). Index audit by (entityType, entityId, occurredAt). Verify actual query plans before adding more. Use pagination bounds, for example default 20 and maximum 100 items. Keep the system small enough to understand before optimizing it.

10 / Correctness: concurrency and retry handling

Preventing double booking

A pre-check followed by an insert without locking is unsafe: two requests can both see availability before either writes. The MVP strategy is serialisation per equipment row using a PostgreSQL row lock. [R17] Inside the transaction: lock equipment; verify ACTIVE; query conflicting bookings; validate policy; insert booking; insert audit and idempotency result; commit.

Every code path that can affect that asset's availability must acquire the same lock first. Locking only the existing booking rows is insufficient when no booking exists yet. Keep transactions short; do not call external services while holding the lock. Use PostgreSQL's normal READ COMMITTED behaviour with the conflict read after the lock is acquired. Test with a real PostgreSQL database, not an in-memory substitute.

Physical collection invariant

After locking equipment, verify no other CHECKED_OUT booking exists, validate ownership/window and transition the requested reservation. A PostgreSQL partial unique index on equipmentId WHERE status = 'CHECKED_OUT' adds protection for physical loans. The time-window rule remains in the service for MVP; an exclusion constraint is an optional additional defence, subject to extension support and careful migration design.

Idempotent creation

The client generates a fresh UUID key for one booking attempt and reuses it only for retries of the same payload. In the transaction, claim the unique idempotency record; concurrent duplicates wait or encounter its uniqueness constraint. If a result already exists with the same normalized payload hash, replay it. If the hash differs, return 409 IDEMPOTENCY_KEY_REUSED. The claim, mutation and saved result must commit or roll back together.

Handle uniqueness conflicts outside an aborted transaction before reading the winning result. Do not keep idempotency only in BFF memory: multiple replicas and restarts would break it. A lost network response after commit must be recoverable by retrying with the same key.

Required tests

Two simultaneous overlapping creations: exactly one succeeds. Adjacent intervals: both succeed. Two concurrent collections for the same asset: at most one CHECKED_OUT. Cancellation versus creation: final state remains consistent. Retry after response loss: one booking and one creation audit event. Time-boundary tests use a fake/injected clock. These tests are part of MVP, not optional polishing.

11 / API contracts and example payloads

Client-facing BFF endpoints

GET /api/v1/me; GET /api/v1/equipment; GET /api/v1/equipment/{id}; GET /api/v1/equipment/{id}/availability?startAt=...&endAt=...; POST /api/v1/bookings; GET /api/v1/bookings/mine; GET /api/v1/bookings/{id}; POST /api/v1/bookings/{id}/cancel; POST /api/v1/bookings/{id}/collect; POST /api/v1/bookings/{id}/return.

Admin routes: GET /api/v1/admin/overview; GET /api/v1/admin/bookings; POST /api/v1/admin/equipment; PATCH /api/v1/admin/equipment/{id}; GET /api/v1/admin/audit?entityType=...&entityId=.... Use the same booking action endpoints with server-enforced admin permissions and an onBehalfReason when applicable.

Spring Boot contracts

Use /v1 paths for backend resource endpoints. Expose resource-oriented queries and commands to the BFF, including /v1/bookings/summary for efficient admin counts. The BFF's mobile/home and admin/overview are compositions, not Java persistence resources. Maintain separate OpenAPI contracts for BFF and Java; generate TypeScript clients/types and check generated code for drift in CI.

Create booking example

```
POST /api/v1/bookings
Authorization: Bearer <access-token>
Idempotency-Key: <uuid>
{
  "equipmentId": "<asset-uuid>",
  "startAt": "2026-10-05T07:00:00Z",
  "endAt": "2026-10-05T10:00:00Z"
}
```

A successful response is 201 with id, equipment summary, startAt, endAt, status RESERVED and allowedActions. The API must derive userId from the validated identity, never this payload. Dates above represent 09:00 to 12:00 in the initial office timezone.

Errors and list responses

```
{
  "code": "BOOKING_CONFLICT",
  "message": "This asset is reserved for part of that time.",
  "traceId": "<correlation-id>",
  "fieldErrors": {}
}
```

List responses use items, page, pageSize and total. Define deterministic ordering, maximum page size and inclusive/exclusive date semantics in OpenAPI. Do not expose the identity of the conflicting borrower to an employee.

12 / Authentication and authorization, step by step

Recommended MVP identity design

Use a workforce Microsoft Entra tenant with test accounts. An Azure subscription grants resource access; an Entra tenant holds identities. They are related but not interchangeable. Creating app registrations and granting consent may need a tenant administrator. If company permissions are unavailable, use an approved sandbox tenant; do not deploy a mock identity service publicly.

Register four logical applications: web client, mobile client, BFF API and Java API. Expose a delegated scope on the BFF and a delegated scope on Java. Configure the BFF to request Java's scope through the on-behalf-of flow. Assign EMPLOYEE/ADMIN app roles on the Java API to test users; Java is the authority for those application roles. Record tenant consent and redirect URI steps separately from resource deployment. [R4]

User request sequence

1. Web/mobile uses authorization code with PKCE to sign the user in and obtain a BFF-audience access token. Neither public client contains a client secret.
2. BFF validates signature, issuer, audience, expiry and delegated scope. It uses the user's access token as an assertion in Entra's on-behalf-of flow to request a Java-audience token.
3. BFF authenticates that exchange as a confidential client using a server-held credential. Keep it in Key Vault, rotate it and never put it in a mobile or browser build. Managed identity access to Key Vault is a different concern from this user delegation flow.
4. Java validates its own token audience/issuer/scope and role claims, then applies ownership checks using tenant/object identifiers. Do not forward a BFF-audience token to Java and disable audience checks to make it work.
5. Java returns the result; BFF shapes it for the client. /me can expose safe capability flags derived from the backend.

Client handling

For web, use a maintained Entra-compatible browser library with PKCE and an in-memory token cache initially. For mobile, use a compatible native/system-browser OIDC implementation and OS-backed secure storage for persisted credentials; confirm its support in the chosen Expo/native build. Do not use ordinary AsyncStorage for refresh tokens. Never send ID tokens as API access tokens.

Restrict BFF CORS to the actual web origin and local development origin. CORS is browser policy, not authentication, and does not protect an API from scripts or mobile callers. Authorize every Java operation. If external staff access is later needed, treat multi-tenancy and guest policies as a separate feature.

13 / Azure fundamentals: what lives where

Subscription, resource group and region

A subscription is the billing and resource-management boundary. A resource group is a named collection of related resources, such as rg-borrowhub-dev. A region is the Azure location where a resource runs. Keep application compute and database near each other; confirm that the chosen region supports your required service tiers and quota before provisioning. South Africa North is a candidate, not an assumed availability guarantee.

A resource is a concrete deployed object: a container registry, a database server or a Container App. Give every resource environment, project and owner tags. Use a single dev environment first; add a separate test resource group later. Shared identity registrations and other bootstrap resources should have their own documented ownership and teardown boundaries.

Control plane versus application traffic

Bicep and the Azure CLI talk to Azure Resource Manager to create/configure resources: this is management or control-plane traffic. A user booking equipment sends HTTPS requests to your BFF: this is application or data-plane traffic. An Azure role allowing someone to deploy the app does not automatically grant them BorrowHub admin access, and a BorrowHub ADMIN role does not let them edit Azure infrastructure.

Resource inventory

Required cloud MVP resources: resource group; Azure Container Registry; Container Apps environment; BFF Container App; Java Container App; manual database migration Container Apps Job; PostgreSQL Flexible Server/database; VNet with appropriate subnets; private DNS; user-assigned managed identities; Key Vault; Log Analytics workspace; Application Insights; Static Web App; cost budget/alerts. Entra app registrations and consent are identity prerequisites.

Later resources: Storage account and private Blob container for photos; scheduled reminder job; notification provider; optional Service Bus and worker. Keep deferred resources out of the first deployment to reduce cost and complexity.

Local-to-cloud mapping

A local Docker image becomes an image in Container Registry. A local running container becomes a Container App replica. Local environment variables become app configuration and secret references. Local PostgreSQL becomes a managed PostgreSQL server. Terminal logs become structured logs searchable in Azure. Docker Compose describes local execution; Bicep describes Azure infrastructure. These are different files for different targets.

14 / Containers, registry and Container Apps

Image versus container

A Dockerfile describes how to build an immutable image containing the runtime and compiled application. A container is a running instance of that image. Build separate images for BFF and Java using multi-stage builds: compile in a builder stage and copy only runtime assets into a smaller final stage. Run as a non-root user and keep credentials out of image layers.

On the Mac, verify the target architecture required by the deployment and explicitly build it, commonly linux/amd64 for the selected environment. A container that works locally on Apple Silicon may otherwise be incompatible with the chosen cloud runtime. Pin base images/dependency versions and use a lockfile or build wrapper.

Azure Container Registry (ACR)

ACR stores your versioned images; it does not run them. CI authenticates to Azure, builds images, tags them with a commit SHA and pushes them to ACR. Deploy by digest where practical so the artifact cannot change underneath an unchanged tag. Avoid relying on latest. Keep the registry admin account disabled; assign push rights to CI and pull rights to the runtime identities using roles appropriate to the registry's permission mode.

Container Apps environment and apps

The environment groups apps with shared networking and logging configuration. Create one BFF app on port 3000 and one Java app on port 8080. Set targetPort to match the process listener. The platform supplies an ingress hostname and can terminate HTTPS. Use external ingress for BFF, internal ingress for Java and the Java internal hostname in BFF configuration. [R1, R2]

Configure CPU/memory, environment variables, health probes, managed identity and replica limits. Begin with small measured allocations and a maximum of two replicas. Choose minimum zero for cost-oriented development if cold starts are acceptable, or one during demonstrations. Java startup time and database connections need monitoring; a second replica creates another connection pool.

Revisions and rollback

A revision is a version of app runtime configuration. Deploying a new image produces a new revision. Use single-revision mode initially; adopt multiple revisions and traffic splitting when you deliberately practise canaries. Rollback means selecting/redeploying the prior healthy image/configuration, not rebuilding an old tag. Application rollback does not reverse a database migration. [R3]

A Container App's writable filesystem is temporary. Never store uploaded equipment photos or PostgreSQL data inside an application container. Environment/secret-only changes have lifecycle details; inspect the resulting revision and running configuration instead of assuming every update restarts the service.

15 / PostgreSQL, networking and DNS

Managed database responsibility

Azure Database for PostgreSQL Flexible Server hosts PostgreSQL and provides managed operational capabilities. You still design tables/indexes, configure access, choose capacity, manage migrations and test restore procedures. Keep durable booking data here, not in BFF memory. Use a dedicated application DB account with limited privileges and a separate migration account where feasible.

JPA uses a JDBC connection pool. Set a conservative pool size, for example five connections per Java replica initially, then check database limits. Connect using TLS with certificate validation; do not disable certificate verification to fix connectivity. Keep credentials out of source and provide them through Key Vault-backed secrets.

Recommended cloud network

Use a VNet-integrated Container Apps environment with a dedicated subnet sized to current environment requirements. Use a separate subnet delegated for PostgreSQL Flexible Server private access. Create and link the PostgreSQL private DNS zone to the VNet. The database DNS name then resolves to an address reachable by your Java app. Follow the selected service's subnet sizing/delegation requirements; do not copy arbitrary CIDRs without checking address ranges. [R7]

A VNet is a private network boundary. A subnet divides its address space. DNS translates a hostname into an IP address. A firewall or network security group controls permitted traffic. A private IP alone does not configure correct name resolution or application credentials.

Important access consequence

A public GitHub Actions runner cannot directly connect to the private database. Run Flyway as a manual Container Apps Job inside the same network, or use an approved private runner. This guide chooses the job. The runner starts the job through Azure's management API, then polls its completion before deploying the application revision. The migration job obtains database credentials securely and uses the matching migration artifact.

Your laptop also cannot automatically reach the private database. Local work uses local PostgreSQL. For exceptional cloud debugging, use an approved VPN/private access path or a temporary job; do not broadly open the database to the internet. A simpler public-firewall learning environment is a separate documented alternative, not the default network in this guide.

Troubleshooting order

If Java cannot connect, inspect hostname resolution, VNet/DNS linkage, subnet rules, TLS configuration, credentials, database name and connection limits in that order. A connection timeout suggests a different class of issue from authentication failure. Never solve either by printing secrets into logs.

16 / Managed identity, Key Vault and configuration

Managed identity in plain language

A managed identity is an Azure-managed identity for a running service. It lets your code or platform request access tokens without embedding an Azure account password. Assign distinct user-assigned identities to BFF, backend and migration job. User-assigned identities can be created before the app, making role assignment and image-pull bootstrapping easier. [R5]

Grant only needed roles at the smallest practical resource scope. The BFF needs registry pull and access to its identity credential secret. Java needs registry pull and its database secret; later it needs limited Blob permissions. The migration job needs its database credential and registry pull. CI needs deployment and image-push access; assigning Azure roles additionally requires appropriate role-assignment permissions, which ordinary Contributor access may not include.

Key Vault

Key Vault stores secrets such as database passwords and the BFF's confidential-client credential. A Container App can reference a Key Vault secret using a managed identity and expose it to the process as an environment secret. The identity must have secret-read permission and network access to the vault. A missing role assignment and a blocked network path are different failures. [R6]

Keep secret values out of Git, .bicepparam files, Docker build arguments, Bicep outputs and logs. Create secret values through a secure bootstrap workflow or protected CI input. Bicep declares the vault, identity, permissions and references. It can accept secure parameters when needed, but that does not make putting plaintext values into committed parameter files safe.

Configuration contract

BFF non-secrets: PORT, JAVA_BASE_URL, ENTRA_TENANT_ID, BFF_AUDIENCE, JAVA_SCOPE, ALLOWED_WEB_ORIGINS. BFF secret: OBO confidential-client credential. Java non-secrets: server port, datasource URL, token issuer/audience, office timezone and policy limits. Java secret: application DB credential. Migration job secret: migration DB credential.

Frontend configuration contains only public values: BFF URL, tenant/client IDs and redirect URIs. Anything bundled into a web or mobile app must be treated as visible to users. Build-time environment variables are not a secret store.

Separate identity concerns

Managed identity authenticates the service to Azure resources. Entra delegated user tokens represent the employee performing a booking. Azure RBAC controls infrastructure access. Java application roles and ownership rules control business actions. Do not use one as an accidental substitute for the other.

17 / Web hosting, logs, traces and costs

React web hosting

Build the React application into static HTML/JavaScript/CSS assets and deploy those files to Azure Static Web Apps. Configure SPA navigation fallback so refreshing /bookings/123 serves the app shell. The browser calls the public Container Apps BFF URL directly; this design does not depend on Static Web Apps managed API integration. Configure the exact web origin in BFF CORS and the correct redirect URI in Entra. [R8]

The native app is installed on devices; it is not hosted as a website. Build/distribution tooling produces the mobile application separately. A universal frontend can still produce separate web and native artifacts. Never use localhost as the phone's cloud API URL; on a physical phone localhost refers to the phone itself.

Observability

Log Analytics stores/query-searches logs. Application Insights provides application telemetry such as requests, dependencies, failures and traces when instrumentation is configured. Provisioning these resources alone does not instrument your code. Configure an appropriate supported Azure Monitor/OpenTelemetry integration for each runtime, propagate W3C trace context and test one end-to-end request. [R12]

Structured logs should include timestamp, level, service, environment, route, duration, status and trace ID. Add a dashboard for BFF/Java error rate, latency, restarts, replica count and DB connections. Application audit events remain in the database and answer who changed a booking; operational telemetry answers why a request was slow or failed.

Cost control

There is no promised free deployment or fixed monthly estimate in this specification. Price depends on region, tier, minimum replicas, database uptime, registry, networking, storage and log volume. Before provisioning, price the exact selected resources with the Azure calculator and get the work/sandbox budget agreed. A managed database may cost money while the APIs are idle.

Set budget alerts and tag every resource. Budget alerts notify; they are not a hard spending cap unless separate automation enforces a response. Limit max replicas, log retention and sampling; avoid recording successful health probes at high volume. Keep dev-only resources easy to identify and tear down. [R10]

Operational acceptance

Demonstrate finding a deliberately failed booking request by trace ID. Show the deployed commit/image digest. Confirm that restarting Java preserves data. Document which resources keep charging when apps scale to zero. Before deleting an environment, identify database data/backups and any shared resources that must be retained.

18 / Bicep: infrastructure as code in practice

What Bicep does

Bicep is a declarative language for Azure resources. You describe the desired resource names, properties and relationships. Azure Resource Manager orchestrates creation and updates. Bicep references let resources consume outputs from others, such as a Container App using an environment ID. Bicep does not compile Java, build Docker images, publish frontend assets or apply database migrations; CI coordinates those separate actions. [R9]

Proposed file organisation

infra/main.bicep composes modules. modules/network.bicep owns VNet/subnets/DNS; registry.bicep owns ACR; identity.bicep owns user-assigned identities and scoped role assignments; secrets.bicep owns Key Vault and references; monitoring.bicep owns logs/telemetry; database.bicep owns PostgreSQL/database; container-environment.bicep owns the environment; apps.bicep owns BFF/Java and migration job; web.bicep owns Static Web Apps; budget.bicep owns supported budget/alert configuration.

infra/parameters/dev.bicepparam holds non-secret dev values. Add test parameters later. Parameters include location, environment name, image digests, replica limits, capacity and tags. Outputs include public BFF/web URLs and resource names needed by CI, never passwords.

Tiny conceptual example

```
param location string = resourceGroup().location
param environmentName string = 'dev'

resource logs 'Microsoft.OperationalInsights/workspaces@2023-09-01' = {
  name: 'log-borrowhub-${environmentName}'
  location: location
  properties: {
    sku: { name: 'PerGB2018' }
    retentionInDays: 30
  }
}

output logWorkspaceId string = logs.id
```

This illustrates parameters, a resource declaration and an output; it is not a complete deployable BorrowHub environment. Validate provider API versions and required properties when implementing. Use deterministic names with a suffix where global uniqueness is required, and deterministic role-assignment GUIDs.

Validation and deployment

Run Bicep compilation/linting, then an Azure what-if preview with the intended parameters. Inspect replacements, deletions and security changes before applying. What-if is a useful preview, not proof that all runtime behaviour or permissions will work. Keep repeat deployments safe; a second deployment with unchanged inputs should not create duplicate resources. Ordinary incremental deployments do not automatically delete resources removed from the source. [R9]

19 / Bootstrap and CI/CD deployment order

One-time prerequisites

Confirm subscription, tenant, deployment permissions, region quotas and budget. Register required resource providers. Create Entra applications, scopes, roles, redirect URIs and consent through an approved identity workflow. Record this as a prerequisite rather than claiming ordinary ARM resource modules automatically complete all tenant administration. Microsoft Graph-based identity automation can be a later addition.

Configure a CI deployment identity with workload identity federation/OIDC to Azure, avoiding a long-lived Azure login secret. Separate dev from future production identities. Document any permissions the bootstrap operator needs to create role assignments. Static Web Apps asset publishing may use its service-specific deployment credential; if so, store it as a protected CI secret and rotate it. OIDC for Azure login does not automatically replace every service's deployment mechanism. [R13]

Avoid the first-deploy circular dependency

1. Deploy foundation modules: resource group/bootstrap scope, identities, registry, VNet/DNS, logging, vault, database and Container Apps environment.
2. Securely populate required secrets and configure database application/migration roles. Allow for RBAC propagation and verify least-privilege access.
3. Build/push BFF, Java and migration images to the registry. Capture their digests.
4. Deploy the migration job definition with its digest, network configuration and secret references. Start it and wait for successful schema migration.
5. Deploy the Java and BFF apps with image digests, health probes and final configuration. Test internal backend connectivity and public BFF authentication.
6. Build/deploy the React web bundle with the actual BFF URL. Finish redirect URI and CORS configuration for the final web hostname. Produce a mobile development build with its registered redirect URI.
7. Run the cloud smoke journey and record evidence. Do not mark deployment complete when only the resource creation succeeded.

Pull request pipeline

Run formatting/linting, TypeScript checks, Java tests, critical PostgreSQL integration tests, API contract generation/drift check, frontend builds and Bicep lint/build. Validate that container builds succeed. Use a protected deployment workflow for cloud changes; do not give untrusted pull requests deployment secrets.

Merge/deploy pipeline

Use the foundation only when infrastructure changes; otherwise build immutable images, preview/apply configuration, execute backward-compatible migrations, update apps, publish web assets and run smoke tests. Abort on migration failure. Keep the previous successful image digests and deployment metadata.

Rollback rule

Rollback app code only when the database remains compatible. Prefer expand-and-contract migrations: add compatible fields first, deploy readers/writers, then remove old fields in a later release. Destructive schema rollback is a separate recovery operation, not an automatic response to a failed health check.

20 / Local development and repository layout

Monorepo shape

apps/mobile/	React Native employee app
apps/web/	React admin application
services/bff/	TypeScript API composition layer
services/backend/	Java Spring Boot modular monolith
packages/api-client/	Generated/shared TypeScript clients
packages/shared/	Portable schemas and formatting
infra/modules/	Bicep resource modules
infra/parameters/	Non-secret environment parameters
infra/main.bicep	Module composition
contracts/	BFF and Java OpenAPI contracts
docs/	Spec, decisions, progress, runbooks
scripts/	Local and CI helper commands
.github/workflows/	CI/deploy workflows if using GitHub
compose.yaml	Local PostgreSQL and optional services

Choose and pin one Node package manager. Commit its lockfile and the Maven or Gradle wrapper for Java. Keep backend code modular; the repository's services directory does not mean every Java module should be deployed separately.

Local startup workflow

Install the selected Java LTS, Node LTS, package manager, Docker-compatible runtime and mobile build prerequisites. Run PostgreSQL through Compose with a named volume. Copy documented .env.example files into ignored local environment files. Apply Flyway migrations, start Java on 8080, BFF on 3000, React dev server and mobile development build. Document the actual commands in README after scaffolding; do not pretend commands exist before scripts are implemented.

Use local mock auth only for an isolated local milestone; verify the real Entra flow before calling cloud MVP complete. A physical iPhone needs a reachable development BFF address and correct redirect configuration. A secure tunnel may be used under work policy; otherwise use the deployed dev BFF.

Development quality rules

Treat generated clients as generated artifacts. Regenerate them from contracts instead of manually editing. Keep .env, certificates and credentials ignored. Document environment variables with purpose, default and whether secret. Each PR should deliver one reviewable vertical slice with screenshots or request evidence when appropriate.

Seed data

Provide ten assets spanning phones, monitors, adapters and cameras. Seed assets independently of user identities; real Entra users are linked after sign-in. Use fixtures or explicitly marked test identities for automated tests. Do not copy company inventory or staff personal data into the practice project without a reason and permission.

21 / Test strategy and MVP completion gates

Unit and service tests

Java: overlap logic, booking-window limits, state transitions, ownership and admin overrides. Inject time to cover exact boundaries. BFF: aggregation shapes, timeout/error mapping and invalid request rejection. Shared/frontend helpers: date conversions and meaningful conditional behaviour, not tests that merely repeat the implementation.

Integration and contract tests

Run Java against Testcontainers PostgreSQL to test migrations, row locks, unique constraints, idempotency and audit atomicity. Test missing, expired, wrong-audience and wrong-issuer tokens plus role/ownership failures. Verify BFF passes a valid downstream audience through the selected OBO flow in a dedicated environment. Ensure generated API clients match OpenAPI and the actual responses.

User journeys

Web: admin signs in, creates an asset, filters bookings and inspects an audit event. Mobile: employee signs in, books, collects and returns; camera denial is tested only when QR is added. Use Playwright for critical web journeys and an appropriate React Native testing/device automation tool for mobile. Include a manual real-device smoke test for sign-in redirects.

Cloud checks

BFF is reachable over HTTPS; Java is not publicly reachable. Public unauthenticated business API calls fail. Database is private. Container restarts preserve bookings. Migration failures stop deploys. Secrets are absent from repository, image layers and logs. Trace context spans BFF and Java. Database restore procedure is documented and at least exercised before relying on real operational data.

Proposed non-functional targets

For seeded demo data and warm services, target common list/detail responses below one second at the 95th percentile during a modest ten-user test; record the actual dataset/load/tool and result. Treat cold-start latency separately. No production SLA or availability promise is implied. Maintain readable text, visible focus, labelled inputs and non-colour-only status indicators.

Definition of done

A task needs working code, relevant tests passing, API/schema updates where applicable, documented configuration changes and evidence in TASKS.md. MVP needs all MVP features and cloud gates complete, no open critical correctness/security defects, a runnable README, a short demo script and a recorded frontend architecture decision. A plan, generated file or passing build alone is not completion evidence.

22 / Delivery plan with exit criteria

Phase 0 / Decisions and skeleton

Tasks P0-01 to P0-04: confirm tenancy/budget/region; decide separate versus universal frontend through a small spike; pin frameworks/toolchains; establish repo, contracts and task tracker. Exit: both clients render a starter screen, services expose local health endpoints and decisions are recorded.

Phase 1 / Local vertical slice

Tasks P1-01 to P1-05: database schema/Flyway; Java catalogue; BFF catalogue; mobile list/detail; web inventory list. Exit: both apps show the same seeded asset through BFF and Java with no client database access. This milestone may use strictly local demo auth.

Phase 2 / Booking correctness

Tasks P2-01 to P2-05: reservation transaction; idempotency; my bookings/cancel; collection/return; admin inventory/bookings/audit. Exit: full local journey works and concurrency/ownership tests pass. Use explicit fixtures until identity integration is finished.

Phase 3 / Identity and cloud foundation

Tasks P3-01 to P3-05: Entra PKCE/OBO and Java authorization; Docker images; Bicep network/data/identity foundation; Container Apps and migration job; React cloud hosting/mobile dev configuration. Exit: real users can complete a secured cloud booking. Disabling mock auth in the cloud is a required gate.

Phase 4 / Repeatability and release

Tasks P4-01 to P4-04: CI/CD and immutable deploys; observability and cost controls; user/device/cloud tests; README, rollback and demo runbooks. Exit: merge-to-dev pipeline works, trace lookup works, and a teammate can reproduce the setup using the documented prerequisites.

How to estimate

Use the phases as completion gates, not promises about calendar duration. After Phase 1, estimate each remaining task from your actual pace. Authentication, private networking and mobile redirects are the main uncertainty areas. Keep only one implementation task in progress; split a task if it spans multiple independently reviewable PRs.

Recommended first Cursor task

Ask Cursor to inspect the repository, read the supplied spec/tracker, propose the Phase 0 file structure and implement only the agreed skeleton. It should verify local startup and record real commands. Do not ask it to generate every service and cloud resource in one pass: feedback from the first vertical slice should inform later implementation details.

23 / Enhancement roadmap: better day-to-day use

ENH-01 / QR collection and return

Print a QR label containing an opaque asset identifier or app deep link, never a credential. Scan with the mobile camera, resolve the asset and let the employee choose a valid reservation/action. Keep manual tag entry as fallback. A QR proves which label was scanned, not the user's identity or permission.

Implementation: mobile camera adapter, deep-link handling, BFF asset lookup and existing Java action endpoints. Acceptance: unknown/damaged codes, denied camera permission and another user's booking all fail safely. Prerequisite: stable MVP collection workflow. Learning: native permissions and linking.

ENH-02 / Photos and damage reports

Add a private Blob container. Spring Boot authorizes an upload request, issues a narrowly scoped short-lived upload mechanism or streams uploads, then verifies completed uploads before linking metadata. Store blob keys and metadata in PostgreSQL, not the file bytes. Use random object names, size/type limits and private access. Mobile/web receive authorized read access rather than permanent public URLs. [R11]

DamageReport stores booking, reporter, description, photos, severity and resolution state. Admin decides whether to move the asset into maintenance; account for future bookings explicitly. Acceptance: one user cannot fetch another user's private report images, abandoned uploads are cleaned up and invalid files are rejected. Consider malware scanning/quarantine before opening arbitrary uploaded files in a real deployment.

ENH-03 / Reminders and inbox

Add an in-app notification inbox first, then push or email through a chosen provider. A scheduled Container Apps Job queries due/overdue bookings and creates notification records. Schedules run independently of API replicas; an in-memory timer in a scaled-to-zero BFF is unsuitable. Use deduplication keys such as booking + reminder type + due period to avoid repeated sends. [R15]

Acceptance: repeated job execution does not duplicate a reminder; failures can retry; cancelled/returned bookings receive no stale reminder; timestamps are interpreted in the office timezone. Register push tokens per device and remove invalid tokens. Notification text must avoid unnecessary sensitive detail.

ENH-04 / Admin calendar and analytics

Add day/week calendar views, utilisation rates and overdue trends. Build server-side aggregates with defined denominators; exclude maintenance time from utilisation if that is the agreed metric. Keep the accessible table as an alternative. Start with read-only calendar navigation; rescheduling introduces new transactional rules and belongs in a separate task.

24 / Enhancement roadmap: reliability and richer policy

ENH-05 / Offline read access

Cache recently viewed catalogue and the user's bookings on the device, clearly showing when data was last refreshed. Do not promise that cached availability is current. Offline booking/checkout mutations are deferred until an explicit queue, idempotency and conflict-resolution experience exists. Acceptance: users can read cached data while offline, but cannot mistake a queued action for a confirmed reservation.

ENH-06 / Approvals and waitlists

High-value assets may require approval. Add a requested state and define whether it holds capacity; choose one policy before coding. Waitlists require fair ordering and an expiring offer when availability opens. Use transactional promotion and tests for competing acceptances. Do not simply email multiple users that each has a guaranteed slot.

ENH-07 / Maintenance and multi-office support

Add maintenance windows, service history and operational blocking. Define how affected future reservations are notified/resolved. Multi-office support introduces office-specific locations, timezones, permissions and transfer workflows. Add `officeId` consistently to entities and authorization queries; do not treat a UI filter as data isolation.

ENH-08 / Reliable asynchronous events

When reminders/uploads become complex, add a transactional outbox table written with the booking transaction. A publisher sends events to Service Bus; a worker processes them with idempotent handlers, retries and a dead-letter path. Database state and event publishing cannot be assumed atomic across separate services. This is a deliberate post-MVP learning exercise, not a prerequisite for a small app.

ENH-09 / Operational hardening

Add infrastructure drift review, dependency/image scanning, restore drills, stricter network egress, secret rotation checks and environment separation. Consider Key Vault/Storage private endpoints after understanding the DNS and egress consequences. Introduce progressive delivery only with metrics that can judge the new revision.

ENH-10 / Browser session BFF

If you want browser tokens off the frontend, adopt a server-managed web session using Secure, HttpOnly cookies with appropriate SameSite behaviour, explicit CSRF protection and shared session/token storage. Keep native bearer-token flows separate. This changes authentication architecture and scaling requirements; create an ADR and targeted tests before implementation.

Prioritisation rule

Build QR scanning, photos and reminders first because they visibly improve the product and exercise mobile/cloud capabilities. Add offline mutations, microservices or multi-office policy only after the MVP is demonstrated and you can explain the current system's failure behaviour.

25 / Cursor operating contract and progress tracking

Files to keep in the repository

Place PROJECT_SPEC.md, TASKS.md, DECISIONS.md and HANDOFF.md under docs/. Place the supplied borrowhub.mdc under .cursor/rules/. The PDF is a readable reference; the Markdown specification and tracker are Cursor's working context. The rule instructs Cursor to read them, but you should still name the current task in each prompt and verify changes.

Required behaviour each session

Read the specification, current decisions, task statuses and handoff. Inspect the actual repository before assuming that a listed file exists. Select the next unblocked task and mark only that task in progress. Explain the intended change, implement it, run relevant verification and update the task with files changed, exact commands, outcomes and remaining limitations.

Never mark a task complete because code was generated. If Azure access, tenant consent or a secret is missing, record a precise blocker and continue only with independent work. Do not invent resource IDs, endpoints, test results or deployment success. Do not silently switch the frontend strategy, authentication flow or technology stack.

Task status model

TODO, IN_PROGRESS, BLOCKED and DONE. A DONE entry needs evidence. Keep planning notes distinct from observed facts. A task can be locally complete but still awaiting cloud verification; represent that as a separate cloud task rather than claiming full completion.

Decision log

Use an ADR entry with ID, date, status, context, options, decision, consequences and revisit condition. Initial proposals: DEC-01 separate frontends pending spike; DEC-02 Fastify BFF; DEC-03 modular Java monolith; DEC-04 PostgreSQL row-lock strategy; DEC-05 Entra PKCE/OBO; DEC-06 private DB plus migration job. Confirm proposals during Phase 0 instead of treating the document as evidence of team approval.

Handoff format

Record current phase/task, what changed, verification performed, unresolved failures, relevant files, next action and any required user/team decision. Include deployed revision/digest only when actually observed. A new Cursor session should be able to continue without reconstructing the whole chat.

Suggested prompt

Read docs/PROJECT_SPEC.md, docs/TASKS.md, docs/DECISIONS.md and docs/HANDOFF.md. Inspect the repository and identify the first unblocked task. Implement only that task and its necessary dependencies, following the specified architecture. Run relevant checks, record their real results, and update the tracker and handoff. Ask only when a missing decision materially changes scope or requires unavailable credentials/permissions.

26 / Troubleshooting and final demo checklist

Common failures and first checks

401 from BFF: check token expiry, issuer, BFF audience and scope. 401 from Java: check that OBO obtained a Java-audience token rather than forwarding the BFF token. 403: inspect app role assignment and ownership. A successful sign-in does not guarantee API authorization.

Browser-only failure: inspect CORS preflight, exact allowed origin and redirect URI. Phone-only failure: inspect reachable API URL, scheme/redirect configuration and system-browser return handling. Native apps are not governed by browser CORS in the same way.

Container starts but requests fail: check process binding to 0.0.0.0, target port, readiness probe path, image architecture and environment variables. Image pull failure: check image digest, registry permission mode, identity attachment and pull role assignment.

Database timeout: inspect network and DNS. Database authentication error: inspect selected username/database and credential reference without exposing values. Migration job failure: inspect job execution logs and Flyway schema history before retrying. Never delete migration history to hide a failure.

Booking duplicates: check idempotency transaction boundaries and unique key scope. Overlapping bookings: verify every mutation path uses the same asset lock and the overlap query runs after locking. Different counts in UI: check timezone boundaries, cache invalidation and shared summary rules.

Ten-minute demo

1. Explain the two clients, BFF and Java responsibility split.
2. Show an asset in mobile and web backed by the same database.
3. Create a booking and demonstrate a conflict from another user.
4. Demonstrate collection/return and admin audit history.
5. Show that a forbidden request is rejected by the backend.
6. Find one request in telemetry by trace ID.
7. Show Bicep modules and a recent successful deployment with image digests.
8. Explain a rollback and the costs/resources that remain when apps are idle.

Completion report

Summarise implemented MVP feature IDs, test evidence, deployment URLs, known limitations, monthly cost estimate using the chosen configuration and the next enhancement. State explicitly whether frontend universalisation was selected or deferred. Keep credentials, private tokens and sensitive tenant details out of screenshots and the report.

27 / Official references and reading order

These references support platform concepts; product rules and architecture defaults are proposed specifically for BorrowHub. Platform behaviour, service tiers and library versions should be verified again during implementation. Documentation checked on 21 September 2026. Read R1, R9, R4 and R7 first.

[R1] Java on Azure Container Apps: hosting a containerised Java service.

<https://learn.microsoft.com/en-us/azure/container-apps/java-overview>

[R2] Secure Container Apps deployments: ingress and security boundaries.

<https://learn.microsoft.com/en-us/azure/container-apps/secure-deployment>

[R3] Container Apps revisions: deployment versions and traffic management.

<https://learn.microsoft.com/en-us/azure/container-apps/revisions>

[R4] Microsoft identity authentication flows: PKCE and on-behalf-of concepts.

<https://learn.microsoft.com/en-us/entra/identity-platform/msal-authentication-flows>

<https://learn.microsoft.com/en-us/entra/identity-platform/v2-oauth2-auth-code-flow>

[R5] Managed identities in Container Apps.

<https://learn.microsoft.com/en-us/azure/container-apps/managed-identity>

[R6] Container Apps secrets and Key Vault references.

<https://learn.microsoft.com/en-us/azure/container-apps/manage-secrets>

[R7] PostgreSQL networking and private access.

<https://learn.microsoft.com/en-us/azure/postgresql/network/how-to-networking>

<https://learn.microsoft.com/en-ca/azure/postgresql/network/concepts-networking-private>

[R8] Azure Static Web Apps overview.

<https://learn.microsoft.com/en-us/azure/static-web-apps/overview>

[R9] Bicep overview and what-if.

<https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/overview>

<https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/deploy-what-if>

[R10] Azure budgets and optional cost-response automation.

<https://learn.microsoft.com/en-us/azure/cost-management-billing/manage/cost-management-budget-scenario>

[R11] Azure Blob Storage introduction and authorization guidance.

<https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction>

<https://learn.microsoft.com/en-us/azure/storage/blobs/authorize-access-azure-active-directory>

[R12] Azure Monitor OpenTelemetry instrumentation.

<https://learn.microsoft.com/en-us/azure/azure-monitor/app/opentelemetry-enable>

[R13] GitHub Actions authentication to Azure through OIDC.

<https://learn.microsoft.com/en-us/azure/developer/github/connect-from-azure-openid-connect>

[R14] Expo development builds.

<https://docs.expo.dev/develop/development-builds/introduction/>

[R15] Container Apps Jobs: manual, scheduled and event-triggered executions.

<https://learn.microsoft.com/en-us/azure/container-apps/jobs>

[R16] Spring Security JWT resource-server configuration.

<https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html>

[R17] PostgreSQL explicit locking.

<https://www.postgresql.org/docs/current/explicit-locking.html>

[R18] Azure Spring Apps retirement announcement. Use Container Apps for this project.

<https://learn.microsoft.com/en-us/azure/spring-apps/basic-standard/retirement-announcement>